DAA Lab - Experiment 9

Implement 0/1 Knapsack Problem using Dynamic Programming

Dr. Ayush Kumar Agrawal
Assistant Professor, School of Computer Science
UPES, Dehradun

Objective

- Implement the 0/1 Knapsack problem using Dynamic Programming.
- Analyze the maximum profit achievable for a given set of weights and values.
- Compare with the greedy (Fractional Knapsack) approach.

Theory

- In the **0/1 Knapsack Problem**, each item can either be included or excluded.
- Dynamic Programming (DP) approach solves it by building solutions to subproblems.
- Recurrence Relation:

$$K[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ K[i-1][w] & \text{if } weight[i-1] > w \\ \max(value[i-1] + K[i-1][w - weight[i-1]], K[i-1][w]) & \text{otherwise} \end{cases}$$

- The final answer is stored in $K[n][W]$.

Algorithm: 0/1 Knapsack using DP

Knapsack(value[], weight[], W, n)

1. Create DP table K[n+1][W+1].
2. For i = 0 to n:
   - For w = 0 to W:
     - If i=0 or w=0: K[i][w] = 0
     - Else if weight[i-1] $\leq$ w:

       $$K[i][w] = \max(value[i-1] + K[i-1][w - weight[i-1]], K[i-1][w])$$

     - Else K[i][w] = K[i-1][w]
3. Return K[n][W]

C Code - 0/1 Knapsack (Part 1)

```c
#include <stdio.h>

int max(int a, int b) { return (a > b) ? a : b; }

int knapsack(int W, int wt[], int val[], int n) {
    int K[n+1][W+1];

    for (int i=0; i<=n; i++) {
        for (int w=0; w<=W; w++) {
            if (i==0 || w==0)
                K[i][w] = 0;
            else if (wt[i-1] <= w)
                K[i][w] = max(val[i-1] + K[i-1][w-wt[i-1]],
                              K[i-1][w]);
            else
                K[i][w] = K[i-1][w];
        }
    }
```

C Code - 0/1 Knapsack (Part 2)

```
    return K[n][W];
}

int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);

    int val[n], wt[n];
    printf("Enter values: ");
    for (int i=0; i<n; i++)
        scanf("%d", &val[i]);
    printf("Enter weights: ");
    for (int i=0; i<n; i++)
        scanf("%d", &wt[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d", &W);

    printf("Maximum profit: %d\n", knapsack(W, wt, val, n));
    return 0;
}
```

Sample Input/Output

Input:
Number of items = 4
Values = 60, 100, 120, 80
Weights = 10, 20, 30, 40
Capacity = 50
Output:
Maximum profit = 220

Complexity Analysis

- Time Complexity: $O(nW)$
- Space Complexity: $O(nW)$
- Efficient version reduces space to $O(W)$ using a 1D array.

Observation Table

Items	Capacity	Max Profit
3	50	220
4	50	220
5	60	250

Comparison

- **Fractional Knapsack:** Greedy, allows fractional items.
- **0/1 Knapsack:** DP, discrete choices only.
- DP guarantees optimal solution for 0/1 Knapsack.

Viva Questions

1. What is the difference between 0/1 and Fractional Knapsack?
2. How does DP improve efficiency?
3. What is the base case in this DP approach?
4. Why is the time complexity $O(nW)$?

Conclusion

- Dynamic Programming provides an optimal solution to the 0/1 Knapsack problem.
- It efficiently balances time and space to achieve best results.

References

- Cormen et al., *Introduction to Algorithms*.
- Horowitz & Sahni, *Fundamentals of Computer Algorithms*.